

Guía de Google OAuth para TI2

Taller de Integración II y Taller de Integración IV (TI4-16, Parte 1)

Vicente Rivera

Segundo semestre 2026

1. Objetivo y alcance

Esta guía prepara a TI2 para implementar en Semana 2 la autenticación institucional con Google (TI2-3). Es investigación y documentación (TI4-16): TI4 no implementa Backend, Web ni Database en este sprint.

Cubre el flujo OAuth con los dos dominios institucionales, el encaje de Better Auth y Convex, Cloud Console, variables sin valores reales y un checklist. Google Calendar queda fuera del Sprint 1. Se coordina con TI2-3 (proveedor y sesión), TI2-14 (logout y seguridad) y TI2-15 (pruebas y documentación).

2. Google Workspace, OAuth 2.0 y OpenID Connect

Google Workspace es el proveedor de identidad. La universidad administra las cuentas, Google verifica la contraseña y el segundo factor, y nuestro sistema solo recibe una identidad verificada. Nunca vemos ni guardamos la contraseña institucional.

El flujo estándar es Authorization Code con OpenID Connect:

1. la Web redirige a **accounts.google.com** con cliente, scopes y **hd** cuando corresponda;
2. el usuario autentica con Google y aprueba los scopes;
3. Google devuelve un código a nuestro callback;
4. Better Auth intercambia el código por tokens y obtiene el perfil OpenID (**openid**, **email**, **profile**);
5. se crea la sesión y Convex emite su propio JWT para las funciones.

Los scopes del Sprint 1 son solo **openid**, **email** y **profile**. Pedir scopes de Calendar, Drive o correo queda prohibido en esta etapa.

En Google Cloud Console el proyecto necesita un OAuth Client de tipo Web application con:

- JavaScript origins autorizados: el origen de la Web y el Site URL de Convex;
- redirect URI autorizada: la ruta de callback de Better Auth en el Site URL de Convex, equivalente a **/api/auth/callback/google**;
- pantalla de consentimiento con logo, contacto y scopes mínimos;
- pantalla de consentimiento de tipo externo, con test users institucionales mientras la app esté en prueba.

Esta guía no incluye valores reales de clientes, secretos ni URLs de deployment. Solo los nombres de las variables y la forma de las rutas.

3. Consola de Google Cloud paso a paso

Todo lo anterior se configura en `console.cloud.google.com`. Es la parte más propensa a errores porque un dato mal copiado bloquea el login con mensajes crípticos. Sigue este orden y verifica cada paso antes de pasar al siguiente.

3.1. Proyecto

Inicia sesión en la consola con una cuenta Google no institucional: las cuentas institucionales tienen Google Cloud bloqueado. Crea o selecciona un único proyecto del equipo. Un solo proyecto es más simple de administrar y basta para este alcance. Todo lo que sigue vive dentro de ese proyecto: pantalla de consentimiento, credenciales y usuarios de prueba. No es necesario habilitar APIs adicionales en la biblioteca, porque el Sprint 1 solo usa autenticación (**openid**, **email** y **profile**) y no consume ninguna API de Google.

3.2. Pantalla de consentimiento OAuth

Ruta: APIs y servicios, Pantalla de consentimiento. Define lo que ve el usuario cuando Google le pide permiso.

- Tipo de usuario **externo**: es el que corresponde, porque el proyecto vive en una cuenta no institucional y el tipo interno no está disponible. Mientras la app esté en modo de prueba, solo entran los usuarios registrados como test users; registra ahí las cuentas institucionales del equipo.
- Datos obligatorios: nombre de la aplicación, correo de asistencia y dominios autorizados.
- Scopes: solo **openid**, **email** y **profile**. Son scopes no sensibles, por lo que no disparan el proceso de verificación extendida que exigen Calendar o Drive.

3.3. ID de cliente OAuth

Ruta: APIs y servicios, Credenciales, Crear credenciales, ID de cliente OAuth. Tipo: **Aplicación web**.

- Orígenes de JavaScript autorizados: el origen de la Web en desarrollo (localhost con su puerto) y el origen de producción en Vercel.
- URIs de redirección autorizadas: la ruta de callback de Better Auth bajo el Site URL de Convex, con forma equivalente a `https://<deployment>.convex.site/api/auth/callback/google`. Debe coincidir exacta: esquema, mayúsculas y slash final. Cualquier diferencia devuelve `redirect_uri_mismatch`.
- Al crear el cliente, copia el Client ID y el Client Secret. El secreto se muestra una sola vez. Guárdalos fuera del repositorio y lléalos al deployment solo con `convex env set`.

3.4. Errores típicos y su causa

redirect_uri_mismatch: el callback no está registrado o difiere en un carácter. Revisa la lista de URIs en Credenciales.

admin_policy_enforced: el admin del Workspace bloquea apps de terceros para sus usuarios. Lo resuelve el administrador de la universidad, no el código.

invalid_client: secreto incorrecto o cliente eliminado. Regenera el secreto y actualiza el deployment.

Acceso bloqueado en modo de prueba: la cuenta no es test user o la app externa aún no se publica. Agrega la cuenta o avanza la publicación.

4. Dominios institucionales

El proyecto distingue dos dominios:

@alu.uct.cl: estudiantes, incluidos los practicantes.

@uct.cl: docentes, funcionarios y administración.

Better Auth soporta la opción `hd` del proveedor Google para exigir un hosted domain de Workspace. El parámetro `hd` sugiere el dominio en la pantalla de Google y, configurado en el proveedor, hace que Better Auth rechace tokens cuyo claim `hd` no coincida.

El flujo obligatorio usa dos inicios lógicos sobre el mismo OAuth Client de Google, uno por población. Cada uno fija su `hd`, valida el sufijo del email en el servidor y asigna el rol inicial ahí mismo. El parámetro `hd` por sí solo es una sugerencia de UX; la garantía real es validar el dominio en el servidor, nunca solo en el frontend.

Estudiantes: <code>hd "alu.uct.cl" + sufijo "@alu.uct.cl" + Estudiante o Practicante</code> Personal: <code>hd "uct.cl" + sufijo "@uct.cl" + solo cuentas habilitadas</code>

El dominio abre la puerta, la habilitación decide el acceso. El inicio de estudiantes cubre a Estudiante y Practicante: ambos son alumnos y se distinguen por habilitación y asignación (TI2-18 y TI2-28), no por el dominio. El de personal cubre a Profesional, pero un email @uct.cl por sí solo no otorga nada: solo entran las cuentas que nosotros habilitamos (estado de habilitación de TI2-4, matriz de TI2-5). El Administrador es transversal al dominio: inicialmente somos nosotros, con cuentas @alu.uct.cl, así que el rol admin se otorga solo por habilitación explícita, venga del inicio que venga. Un email fuera de ambos sufijos, o sin habilitación, se rechaza en el servidor sin crear sesión útil.

5. Cómo encajan Better Auth y Convex

Better Auth administra registro, login social, sesión y logout. Convex aporta ejecución, base de datos y verificación del JWT en cada función. La integración oficial (@convex-dev/better-auth) hace que Better Auth corra como rutas HTTP dentro del deployment Convex y persista sus tablas en el componente `betterAuth`.

<pre>Web (Vite SPA) +-- authClient.signIn.social({ provider: "google" }) v https://<deployment>.convex.site/api/auth/* (Better Auth sobre Convex) +-- Google OAuth + callback +-- tablas user, session, account, verification v Convex functions verifican ctx.auth.getUserIdentity()</pre>

Los archivos del backend son siempre los mismos:

<pre>convex/ +-- auth.config.ts proveedor JWT de Better Auth +-- convex.config.ts registra el componente betterAuth +-- http.ts expone /api/auth/* en Convex Site</pre>

```
+-- auth.ts createAuth: baseUrl, adapter,  
| socialProviders.google, trustedOrigins  
+-- schema.ts solo dominio; auth vive en el componente
```

Fragmento de forma (sin valores):

```
auth.config.ts:  
  providers: [getAuthConfigProvider()]  
  
convex.config.ts:  
  app.use(betterAuth)  
  
http.ts:  
  authComponent.registerRoutes(http, createAuth)  
  
auth.ts:  
  baseUrl: process.env.BETTER_AUTH_URL  
  database: authComponent.adapter(ctx)  
  socialProviders: { google: { clientId, clientSecret } }  
  plugins: [convex(), crossDomain()]
```

En la Web Vite (SPA, sin Start) el cliente es:

```
import { createAuthClient } from "better-auth/react";  
import { convexClient, crossDomainClient } from  
  "@convex-dev/better-auth/client/plugins";  
  
export const authClient = createAuthClient({  
  baseUrl: import.meta.env.VITE_CONVEX_SITE_URL,  
  plugins: [convexClient(), crossDomainClient()],  
});
```

Y el provider en la raíz:

```
const convex = new ConvexReactClient(  
  import.meta.env.VITE_CONVEX_URL  
);  
  
<ConvexBetterAuthProvider client={convex} authClient={authClient}>  
  <App />  
</ConvexBetterAuthProvider>
```

El login social desde TanStack Router es una llamada al cliente, con `callbackURL` y `errorCallbackURL` del frontend:

```
await authClient.signIn.social({  
  provider: "google",  
  callbackURL: "/",  
  errorCallbackURL: "/login",  
});
```

La sesión reactiva se lee con `authClient.useSession()`. La verdad autoritativa, sin embargo, siempre está en el backend con `ctx.auth.getUserIdentity()` o la lectura del usuario autenticado que expone el componente. La regla es: nunca aceptar un `userId` desde los argumentos para autorizar; la identidad viene del JWT.

6. Autenticación no es autorización

Autenticación: quién es el usuario. La resuelve Better Auth con Google y la entrega como identidad.

Autorización: qué puede hacer. La resuelven los casos de uso y el dominio, en el backend, a partir de esa identidad.

```
Infraestructura
|
+--> Better Auth entrega identidad (quién)
|
v
Aplicación y Dominio evalúan permiso (qué puede)
|
v
Convex ejecuta o rechaza
```

Ocultar un botón en React no es autorización. Toda regla (rol, recurso, pertenencia) debe verificarse en la función Convex antes de leer o mutar. La matriz de permisos vive en TI2-5 y las rutas protegidas en TI2-6 y TI2-20; esta guía solo muestra cómo se conectan con la sesión.

7. De la sesión al portal correcto

El login deja una sesión, pero no decide a qué portal entra el usuario. Esa decisión combina tres piezas, cada una con su dueño:

```
TI2-3 entrega identidad (quién + email verificado)
|
v
TI2-5 entrega rol (qué es: Estudiante, Profesional,
  Practicante o Administrador)
|
v
TI2-6 y TI2-20 navegan (guards que leen sesión + rol
  y redirigen al portal o a denegado)
```

El dominio sugiere, la habilitación decide. Hay dos dominios y cuatro roles: `@alu.uct.cl` anticipa Estudiante o Practicante, y `@uct.cl` anticipa Profesional, siempre que la cuenta esté habilitada. El Administrador queda fuera de esa correspondencia: se otorga por habilitación explícita sin importar el dominio. Navegar solo por `hd` o por sufijo del email lleva al portal equivocado. El guard debe leer el rol que expone el Backend, nunca deducir el destino desde el dominio.

El patrón esperado es:

- el Backend expone una query autenticada que devuelve el rol del usuario actual a partir de `ctx.auth.getUserIdentity()`;
- el guard de TanStack Router (por ejemplo en `beforeLoad`) exige sesión válida y rol permitido;
- sin sesión redirige a `/login` conservando la ruta de retorno;
- con sesión pero sin rol suficiente muestra acceso denegado;
- el guard es UX, no seguridad: la función Convex vuelve a verificar el permiso antes de responder.

El detalle de la matriz queda en TI2-5 y el de cada portal en TI2-6 y TI2-20. Esta sección existe para que TI2 no diseñe el login aislado del portal que lo continúa.

8. Variables, callbacks y redirects

Solo nombres. Ningún valor real pertenece a esta guía ni al repositorio.

Variable	Uso
GOOGLE_CLIENT_ID	OAuth Client de Google, solo servidor
GOOGLE_CLIENT_SECRET	Secreto OAuth, solo servidor
BETTER_AUTH_SECRET	Firma de sesiones, solo servidor
BETTER_AUTH_URL	Base URL de Better Auth en Convex
BETTER_AUTH_TRUSTED_ORIGINS	Orígenes extra permitidos
VITE_CONVEX_URL	Cliente Convex, pública
VITE_CONVEX_SITE_URL	Base de /api/auth/*, pública

Reglas:

- las secretas se configuran con `convex env set` en el deployment, nunca con prefijo `VITE_` ni en el bundle;
- las públicas `VITE_` son visibles para cualquiera; solo localizan servicios;
- `trustedOrigins` debe incluir el origen de desarrollo Vite (puerto de dev), el origen de producción en Vercel y el Site URL de Convex; se extiende por entorno con `BETTER_AUTH_TRUSTED_ORIGINS`;
- el callback Google es la ruta `/api/auth/callback/google` bajo el Convex Site URL; el frontend solo define `callbackURL` y `errorCallbackURL` de navegación post-login.

Archivos de ejemplo sin secretos: `apps/web/.env.example` con `VITE_CONVEX_URL` y `VITE_CONVEX_SITE_URL`; locales en `.env.local` fuera de Git.

9. Checklist para que TI2 comience en Semana 2

1. Completar la sección de Consola de Google Cloud: cuenta no institucional, pantalla de consentimiento externa con test users, OAuth Client Web con redirect al Site URL de Convex y scopes `openid`, `email`, `profile`.
2. Implementar los dos inicios con `hd` (`alu.uct.cl` y `uct.cl`), validación de sufijo y rol inicial en el servidor.
3. Levantar `convex/auth.config.ts`, `convex.config.ts`, `http.ts` y `auth.ts` según la estructura oficial.
4. Configurar secretos solo con `convex env set` y publicar `.env.example` sin valores.
5. Conectar la SPA Vite con `authClient`, `convexClient`, `crossDomainClient` y `ConvexBetterAuthProvider`.
6. Implementar login y logout con `signIn.social` y `signOut`, más lectura con `useSession`.
7. Verificar en una query que `ctx.auth.getUserIdentity()` distinga autenticado de anónimo.
8. Registrar decisiones y dudas como comentarios en TI2-3 para coordinar con TI4.

10. Referencias oficiales

- Better Auth, Google social provider y opción `hd`: <https://better-auth.com/docs/authentication/google>
- Better Auth, uso de `signIn.social`: <https://better-auth.com/docs/basic-usage>

- Better Auth con Convex (guía marco React): <https://labs.convex.dev/better-auth/framework-guides/react>
- Better Auth con Convex (integración): <https://better-auth.com/docs/integrations/convex>
- Convex, cliente React: <https://docs.convex.dev/client/react/overview>
- Convex en hosting Vercel: <https://docs.convex.dev/production/hosting/vercel>
- Google Identity, OAuth 2.0 y OpenID Connect: <https://developers.google.com/identity/openid-connect/openid-connect>
- Google Cloud, configuración de pantalla de consentimiento y Client ID: <https://developers.google.com/identity/protocols/oauth2>
- Google Cloud, página de clientes OAuth (crear credenciales): <https://console.developers.google.com/auth/clients>
- Google Cloud, tipos de usuario y pantalla de consentimiento: <https://support.google.com/cloud/answer/10311615>
- TanStack Router con Vite: <https://tanstack.com/router/latest/docs/installation/with-vite>